Predictive maintenance case study: technical review

Role-play review: an AI reviewer acting as a rotating-equipment data scientist assesses Richard Lobo Sapan's portfolio project. Results regenerated 2026-10-23. Code: github.com/richardy-lobo-sapan/pdm-case-study

87 / 100

was 73 as submitted

Strong hire signal. Leakage control, cost-based decisions and honest uncertainty already met industry practice. The review fixed a cost sweep that never tested its low end and a baseline that flattered LightGBM, then ran the pipeline on harder data (FD004). The approach held there, and the remaining weakness is now clear: on equipment that runs in several operating conditions, the confidence ranges need calibrating per condition.

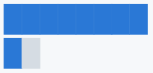
Verdict

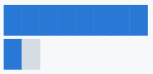
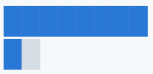
The project does what most portfolio projects skip. It validates by engine, chooses the alert threshold by cost, puts an honest range on every number and serves the model behind an API that reproduces the offline results exactly. The two things I would have held against it in an interview are fixed, and the score rose from 73 to 87 through work that can be checked in the repository, not through re-grading.

What keeps it below 90: the data is simulated, the ranges are not yet calibrated per operating condition, and the API has no authentication. The first needs a pilot on real equipment; the other two are a few days of work.

Score breakdown

Each area is scored out of 10 and weighted by what a field data science team needs from a hire. The total is the weighted average, out of 100.

Area	Weight	Before	Now		Evidence
Business framing	15%	9	**9**		Answer-first story, decision tied to cost, stand-in data and assumed costs labelled everywhere.
Validation rigour	15%	9	**9**		Engine-grouped CV everywhere, settings chosen on other engines, no-look-ahead tests, API parity test.
Modelling and uncertainty	15%	7	**9**		Fair baseline, seed spread, coverage by band, and an ablation showing the model reads wear, not age.

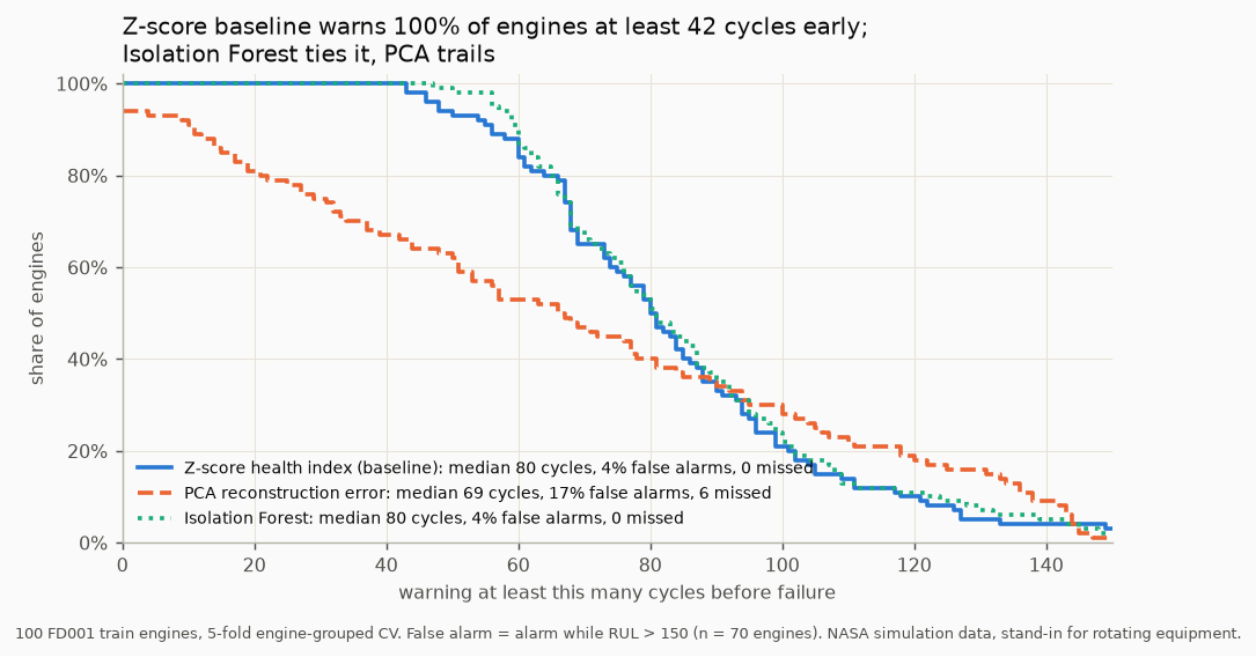
Early warning	10%	8	**9**		Simple index beats PCA, ties Isolation Forest, and still warns every engine on FD004.
Decision and cost analysis	15%	6	**9**		Honest cost sweep with a break-even, nested bootstrap, saving holds on FD004.
Engineering and deployment	15%	7	**9**		API, Docker, MLflow, input-range check, CI on every push, public repo. No authentication yet.
Realism for field data	15%	5	**7**		FD004 (6 conditions, 2 fault types) handled, but its ranges under-cover. Still simulated data.
Total	100%	73	87		

How the score moved

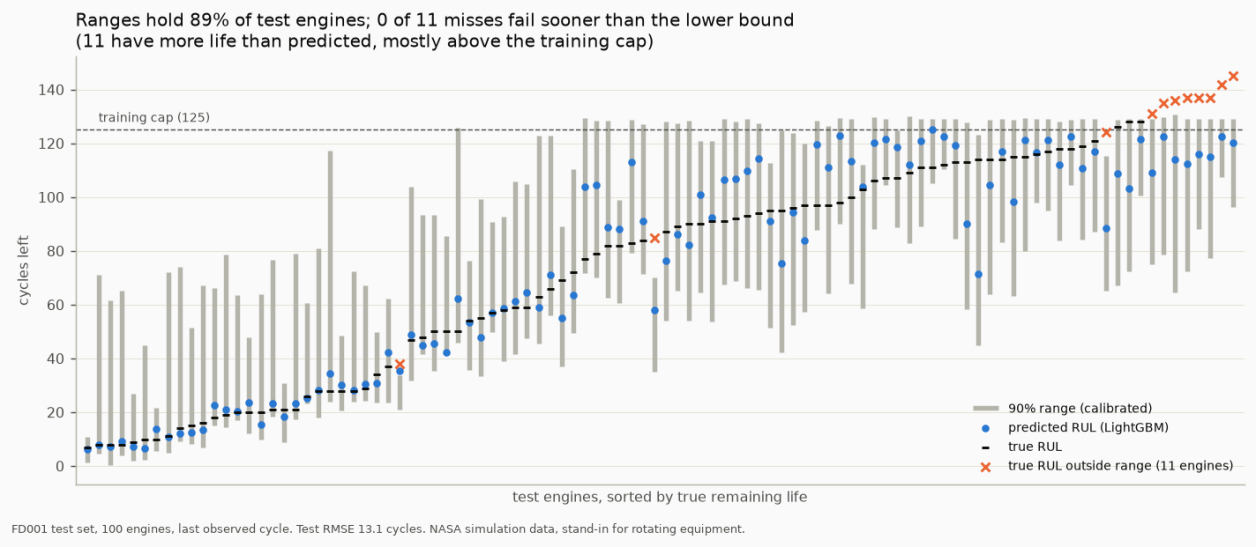
Step	What changed	Score
As submitted	Starting point	**73**
Review fixes	Honest cost sweep, fair baseline, API input check, coverage by band, seed spread	**79**
Public repo and CI	Tests run on every push; anyone can open the repo	**80**
Ablation and nested bootstrap	Model reads wear, not age; saving range redone with settings re-chosen	**83**
FD004 robustness check	Harder data with 6 operating conditions and 2 fault types	**87**

What is strong

- Leakage control is built in. Every split groups by engine, and the cost-simulation settings for each engine are chosen on other engines. Tests check both.
- The early-warning result is simple and credible. A per-engine z-score index warned 100 of 100 engines, 90% of them at least 55 cycles ahead, with 4% of engines raising a false alarm. Choosing it over fancier models was the right call.
- Uncertainty is calibrated and checked on unseen data. Raw quantile ranges held 75% of test outcomes; after conformal calibration 89%, against a 90% target.
- Deployment matches the analysis. The API reproduces the offline test prediction exactly (a test fails otherwise), and the LLM field note is rejected if it contains any number the model did not produce.



Early warning on FD001: share of engines warned at least N cycles before failure, held-out engines only.



Remaining life on the FD001 test set: every miss of the 90% range is an engine with more life than predicted.

Findings and fixes

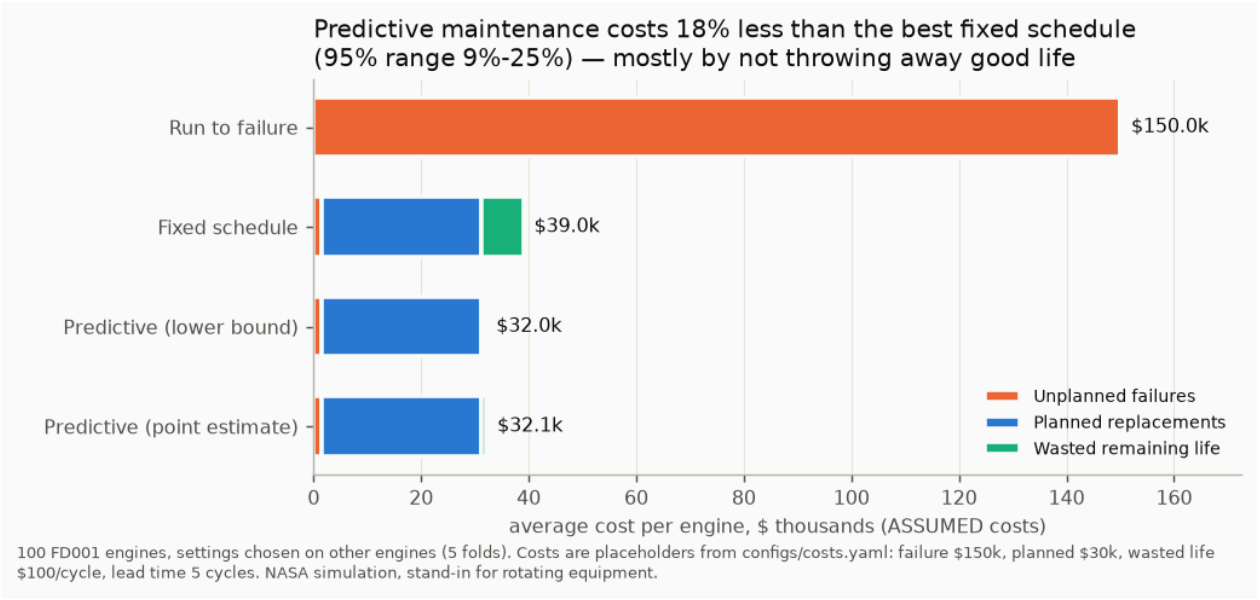
Ranked by severity. Eleven of twelve findings are fixed; the twelfth came out of the FD004 check.

#	Severity	Finding	Status
1	High	Repository was private: the public link returned 404, so reviewers saw nothing.	**Fixed:** repository is public, with a test badge in the README.
2	High	The failure-cost sweep held downtime fixed, so its "1:1" point was really 2.3:1 and the low end was never tested.	**Fixed:** the whole failure cost is scaled. Predictive pays from 1.1:1.
3	Medium	The only linear baseline used current readings (test RMSE 21.9), so LightGBM looked 40% better.	**Fixed:** ridge on the same features scores 17.7; LightGBM's own gain is 26%.

4	Medium	The API accepted any sensor values, so a stuck sensor still got a confident answer.	**Fixed:** /predict flags readings outside the training range; the dashboard warns.
5	Medium	Only FD001 (one operating condition, one fault type).	**Fixed:** FD004 robustness check with 6 conditions and 2 fault types.
6	Medium	No continuous integration: tests ran only on one laptop.	**Fixed:** GitHub Actions runs the tests on every push.
7	Medium	FD004 ranges under-cover: 84% of test rows instead of 90%, and 15 test engines failed before their lower bound.	**Open:** calibrate ranges per operating condition before a multi-condition pilot.
8	Low	Coverage near failure was analysed in a notebook but not saved by the pipeline.	**Fixed:** coverage per remaining-life band in results.json.
9	Low	Results came from one random seed.	**Fixed:** test RMSE 13.0 ± 0.29 over 5 seeds.
10	Low	The bootstrap kept each engine's cost fixed, ignoring uncertainty in choosing the threshold.	**Fixed:** nested bootstrap gives 7% to 28%.
11	Low	Engine age carried a large share of the model's push; in the field, age is often unknown.	**Fixed:** without age, test RMSE is 14.8 (vs 13.1).
12	Low	The API has no authentication or rate limit.	**Open:** fine for a demo; add before any pilot.

What the fixes showed

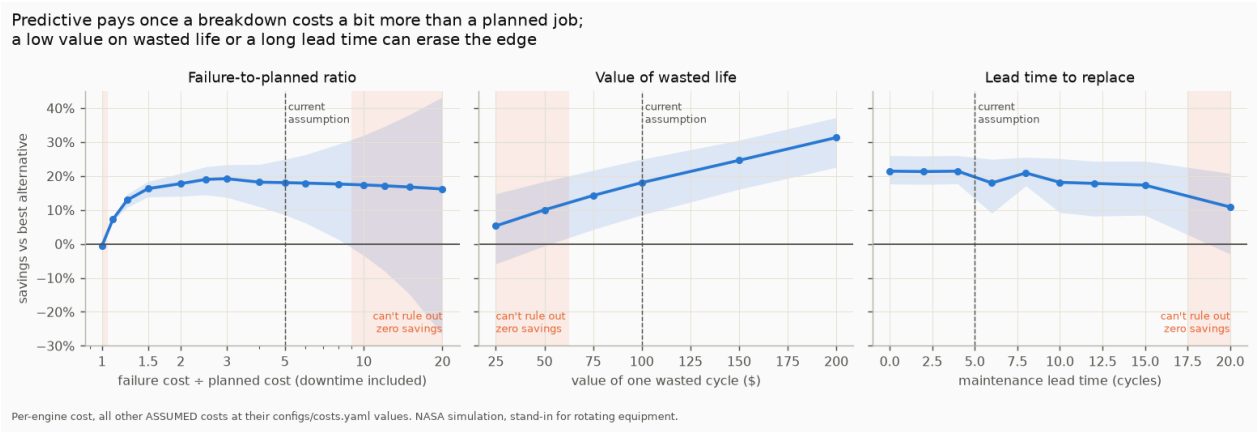
The money result holds under a stricter check



Average cost per engine on held-out FD001 engines. All costs are assumptions from configs/costs.yaml.

Predictive replacement costs 18% less per engine than the best fixed schedule. The original 95% range was 9% to 25%; redoing the threshold choice inside every bootstrap resample widens it to 7% to 28%, still above zero.

Where predictive stops paying



Savings against the best alternative as one assumption varies; shaded band is the 95% range.

With downtime included, predictive maintenance gives no saving at a 1:1 failure-to-planned cost ratio (-0.6% against running to failure) and pays from 1.1:1. From 10:1 upward the range includes zero again, because one unlucky failure then decides the result. The assumed costs sit at 5:1.

Failure ÷ planned cost	Best alternative	Saving	95% range
1 : 1	run to failure	-0.6%	-0.7% to -0.4%
1.1 : 1	run to failure	7.2%	6.9% to 7.5%
1.5 : 1	fixed	16.3%	13.9% to 18.4%
2 : 1	fixed	17.8%	14.1% to 20.8%
5 : 1	fixed	18.1%	8.6% to 24.9%
10 : 1	fixed	17.4%	-3.4% to 32.0%
20 : 1	fixed	16.2%	-26.7% to 43.3%

LightGBM against a fair baseline

Model (FD001 test set, last cycle of 100 engines)	RMSE (cycles)	NASA score (sum)
Predict the average	43.1	33,629
Linear, current readings	21.9	1,314
Ridge, same rolling features	17.7	458
LightGBM, rolling features	13.1	236
LightGBM without engine age	14.8	not computed

LightGBM still wins clearly, 26% lower error than a linear model given the same information, and removing engine age costs little, so the model reads degradation rather than age.

Coverage where decisions are made

True remaining life (cycles)	Rows	Coverage	Truth below lower bound	Mean width
[0, 10]	1,100	99.7%	0.3%	28.1
(10, 25]	1,500	97.5%	1.7%	34.8
(25, 50]	2,500	89.4%	8.5%	47.9
(50, 75]	2,500	87.2%	10.1%	58.9
(75, 100]	2,500	89.6%	7.5%	57.1
(100, 125]	2,500	90.9%	4.8%	48.6

Close to failure, where the replace decision is made, the range is conservative: it holds 99.7% of outcomes within 10 cycles of failure. Mid-life it under-covers slightly.

Harder data: FD004

Metric	FD001	FD004
Operating conditions / fault types	1 / 1	6 / 2
Training / test engines	100 / 100	249 / 248
Engines warned before failure	100 of 100	249 of 249
90% of engines warned at least	55 cycles ahead	44 cycles ahead
Engines with a false alarm	4%	2%
Test RMSE, engines within 125 cycles of failure	11.9	13.5
Test RMSE, all test engines	13.1	25.6
Range coverage, test rows within 125 cycles	92%	84%
Test engines failing before the lower bound	0	15
Saving vs best fixed schedule	18%	28%
95% range, settings re-chosen	7% to 28%	22% to 30%

With each sensor normalized within its operating condition, early warning and the money result carry over. The higher overall RMSE is mostly the training cap: 67 of 248 test engines have more than 125 cycles left, which a capped model cannot reach by design. The real weakness is calibration: ranges fitted on the training engines cover fewer test rows than promised, so a pilot on multi-condition equipment should calibrate per condition first.

Next steps

1. Calibrate the ranges per operating condition (conformal calibration done separately for each condition) and re-check FD004 coverage.
2. Add authentication to the API before it is used outside a demo.
3. Pilot on one fleet's real sensor history and finance-approved costs, with this project's results as the bar to clear.
4. In interviews, lead with the money slide and the break-even: "predictive pays once a failure costs more than a planned job, and the saving comes from not wasting parts".

All costs in this project are assumptions in configs/costs.yaml, and NASA C-MAPSS is simulation data standing in for rotating equipment. Nothing here describes any company's operations or costs.